

COMP101

Introduction to Programming

2025-26

Lecture-26

2-d List Output

Numbers

Output a 2-d list to 'look like' a 2-d list

Use nested for loops

```
list_3x4 = [  
    [1,2,3,4],  
    [5,6],  
    [7,8,9]  
]
```

Outer loop iterates through the row number;
Inner loop iterates through columns

```
for row in range(len(list_3x4)):  
    for column in range(len(list_3x4[row])):  
        print (list_3x4[row][column], end=" ")  
    print()
```

1 2 3 4
5 6
7 8 9

end = " " puts the o/p after the previous o/p and on the same line
print() starts a new line after each row (and its columns) have been o/p
indent print() correctly to avoid o/p down or across the screen

Output a 2-d list: Similar methods – same output

```
list_3x4 = [ [1,2,3,4], [5,6,7,8], [9,10,11,12] ]
```

```
for row in list_3x4:  
    for element in row:  
        print (element,end = " ")  
    print()
```

```
for row in list_3x4:  
    for column in row:  
        print (column,end = " ")  
    print()
```

```
for i in list_3x4:  
    for j in i:  
        print (j, end = " ")  
    print()
```

```
for row in range (len(list_3x4)):  
    for column in range(len(list_3x4[row])):  
        print (list_3x4[row][column], end=" ")  
    print()
```

All use a nested for loop

Output a 2-d list

```
num_list = [ [1,2,3], [4,5,6], [7,8,9] ]
```

```
for row in num_list:
```

```
    for column in row:
```

```
        print (column,end = " ")
```

```
    print()
```

use single or double quotes

#a space between the quotes puts a space between the numbers in each row

#note indentation of last print()

Output individual rows and columns

row	[0]	[1]	[2]
	[1, 2, 3],	[4, 5, 6],	[7, 8, 9]]
column	[0] [1] [2]	[0] [1] [2]	[0] [1] [2]

```
list_out = [ [1,2,3], [4,5,6], [7,8,9] ]
```

```
print (list_out[1])           #output row-2
```

```
[4, 5, 6]
```

```
print (list_out[1][2])       #output row-2, column-3
```

```
6
```

```
print (len(list_out[2]))     #output number of values in row-3
```

```
3
```

Remember the offset
– indexes start at 0

List names

Information only

- how the list is populated gives its name

#'dense list'

#most or all elements have a non-zero value

dense_list = [[1,2,3], [4,5,6] , [7,8,9]]

1	2	3
4	5	6
7	8	9

#'sparse list'

#most elements have a zero value

sparse_list = [[1,0,3], [0,0,6], [7,0,0]]

1	0	3
0	0	6
7	0	0

Output diagonal population

Information only

#create 4x4 array – list comprehension

```
list_diag_4x4 = [[0 for row in range (4)] for col in range (4)]
```

#populate the diagonal with 1's from [0][0] to [3][3]

```
for i in range(4):
```

```
    list_diag_4x4[i][i] = 1
```

#output the list – using a nested for

```
for row in list_diag_4x4:
```

```
    for column in row:
```

#without end=" ", output is one line down the screen

```
    print(column,end=" ")
```

#exit inner loop – print() stops o/p bring one line across the screen

```
    print()
```

This method follows the 'normal' approach to list initialisation:

- a) a 'create' part
- b) a 'populate' part
- c) an 'output' part

```
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
```